Week05

- stacks
- queues

CS136

Docs FAQ HelpHoursCalendar LectureRecordings Midterm2024 Final2024 PracticeQuestions MorePracticeQuestions

✨ No-Laptop Wednesday! ✨

✨ Lida and Aidan ✨ will be covering lecture on Friday (Tut05)

✨ WARMUP ✨

```
// What does this print?

int[] array = { 2, 3, 4 };

ArrayList<Integer> list = new ArrayList<>();
for (int i = 0; i < array.length; ++i) {
    for (int repetition = 0; repetition < 3; ++repetition) {
        list.add(array[i]);
    }
}

PRINT(list);

for (int i = 0; i < list.size() / 2; ++i) {
    int j = (list.size() - 1) - i;
    int tmp = list.get(i);
    list.set(i, list.get(j));
    list.set(j, tmp);
}

PRINT(list);

for (int i = 0; i < list.size(); ++i) {
    list.set(i, list.get(list.get(i))); // <- Read carefully!
}

PRINT(list);
```



FlipBook

were array lists a good choice for implementing a Flip Book?

(how) could you have done it with just arrays?

well, we didn't know how many...

- pages in the flipbook?
- strokes in each page?
- points in each stroke?

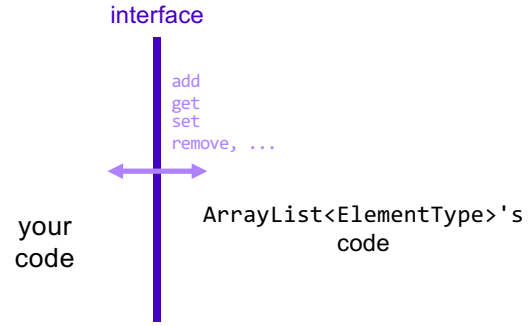
"Alternative" 1: just use arrays, but resize them yourself when needed (essentially, implement the functionality of an array list without actually having a class)

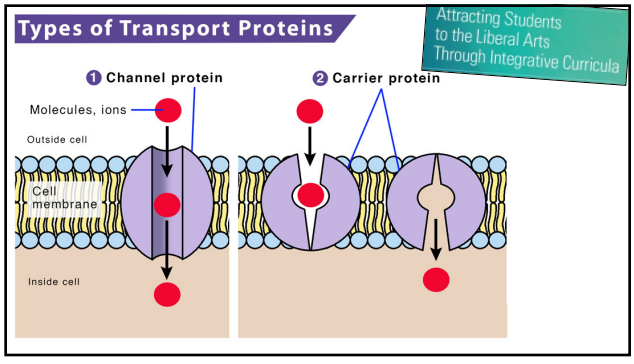
"Alternative" 2: 🐘 use a BIG multi-dimensional array

```
// ~30,000,000 elements
Point[][][] flipbook = new Point[64][512][1024];
// NOTE: we also need all these counter variables
// can't just call .size() like before!
int numPages = 0;
int[] numStrokes = new int[64];
int[][] numPoints = new int[64][512];
```

these alternatives are probably kind of trash;
array lists considered helpful 😳 👍

interface





interface

- a data structure's **interface** (**API**) or **abstract data type** is a set of functions a data structure must have
  - a **list** has...
    - **get**
    - **set**
    - **add**
    - **remove**
    - etc.
- a **data structure** is a specific **implementation** (code that does the thing) of that interface
  - an **array list** implements the list interface using an array
  - 🔮 a **linked list** implements a list using nodes that refer to nodes

you can get (very) formal about this

The abstract list type L with elements of some type E (a monomorphic list) is defined by the following functions:

- nil: $() \to L$
- cons: $E \times L \to L$
- first: $L \to E$
- rest: $L \to L$

with the axioms

- first (cons (e, l)) = e
- rest (cons (e, l)) = l

for any element e and any list l. It is implicit that

- cons (e, l) ≠ l
- cons (e, l) ≠ e
- cons (e_1, l_1) = cons (e_2, l_2) if $e_1 = e_2$ and $l_1 = l_2$.

Note that first (nil ()) and rest (nil ()) are not defined.

These axioms are equivalent to those of the abstract stack data type.



i typically won't.

just know there is a difference between

an **interface** ("a list")
and
a specific **implementation** (ArrayList<Element>)

stack

analogy

a stack is like a helicopter pilot
with a socially unacceptable hobby

`stack.push(🐨)`



stack.push(🐻)



stack.push(🐷)



stack.pop()



stack.pop()



stack.push(🐮)



stack.pop()



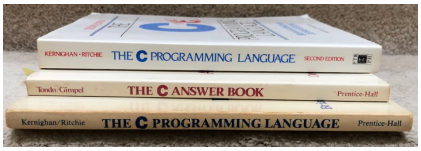
`stack.pop()`



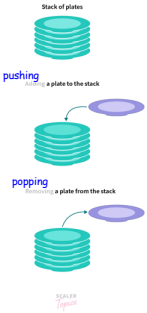




stack







Our Standard Stack Analogy Is Wrong

In search of a better stack metaphor

October 7, 2021

The most common way we introduce the stack data structure is by saying it's like a spring-loaded plate dispenser, the kind found in a cafeteria:



Indeed when I looked up stacks in an old computer architecture textbook[1] of mine I found:

> stacks are frequently described by the way plates are stored and used in a cafeteria. New plates are added to the top of the stack, or pushed, and plates already on the stack move down to make room for them.

Often we augment the comparison with a diagram showing the members of the stack being pushed down or popped up[2]:

Using a Stack







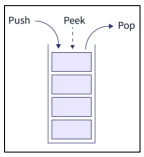
stack interface

```
// Push (add) a new element to the top of the stack.
void push(ElementType element);

// Pop (remove) the top element of the stack.
// and returns it.
ElementType pop();

// Peek (look) at the top element of the stack
// (without removing it) and return it.
ElementType peek();

// Returns the number of elements currently in the stack.
int size();
```



what does this print?

```
Stack<Integer> stack = new Stack<>();
stack.push(2);
stack.push(3);
stack.push(4);
stack.push(5);
PRINT(stack.pop());
PRINT(stack.peek());
PRINT(stack.size());
PRINT(stack);
```

```
Stack<Integer> stack = new Stack<>();
stack.push(2);
stack.push(3);
stack.push(4);
stack.push(5);
PRINT(stack.pop());  // 5
PRINT(stack.peek()); // 4
PRINT(stack.size()); // 3
PRINT(stack); // BOTTOM[ 2, 3, 4 ]TOP
```

queue

q

😬

analogy

a queue is like a line of very polite people patiently waiting

queue.remove()



queue.remove()



queue.add( )



queue.add( )



queue.remove()



queue.remove()





queue



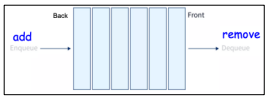




SCALER Topics

queue interface

- `// Add (enqueue) a new element to the back of the queue.`
  `void add(ElementType element);`
- `// Remove (dequeue) the front`
  `// element and return it.`
  `ElementType remove();`



- `// Peek (look) at the front element (without removing it)`
  `// and return it.`
  `ElementType peek();`
- `// Returns the number of elements currently in the queue.`
  `int size();`

what does this print?

```
ArrayDeque<Integer> queue = new ArrayDeque<>();
queue.add(7);
queue.add(8);
queue.add(9);
PRINT(queue.remove());
PRINT(queue.peek());
PRINT(queue.size());
PRINT(queue);
```

```
ArrayDeque<Integer> queue = new ArrayDeque<>();
queue.add(7);
queue.add(8);
queue.add(9);
PRINT(queue.remove()); // 7
PRINT(queue.peek());   // 8
PRINT(queue.size());   // 2
PRINT(queue); // FRONT[ 8, 9 ]BACK
```

stacks & queues

(why are these in the same lecture)

stack interface

```
// Push (add)    a new element to the top  of the stack
void push(ElementType element);

// Remove (pop)     the top   element of the stack
// and returns it.
ElementType pop();

// Peek (look) at the top   element of the stack
// (without removing it) and return it.
ElementType peek();

// Returns the number of elements currently in the stack
int size();
```

queue interface

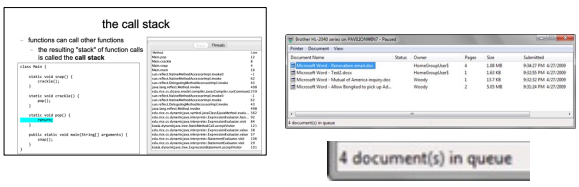
```
// Add (enqueue) a new element to the back of the queue
void add(ElementType element);

// Remove (dequeue) the front element of the queue
// and return it.
ElementType remove();

// Peek (look) at the front element of the queue
// (without removing it) and return it.
ElementType peek();

// Returns the number of elements currently in the queue
int size();
```

💭 what are some examples of stacks and queues in...
- real life?
- computers?
- computer science?



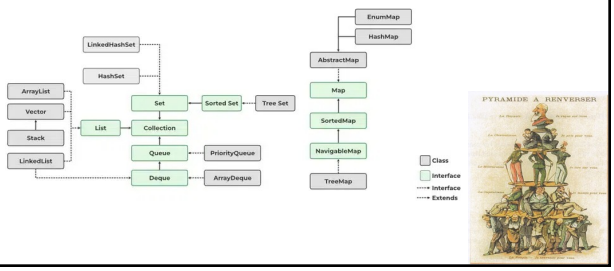
[input buffering in Safari's address bar]

Stack, Queue, and ArrayList
(philosophies of Java vs Python)



why not just use ArrayList as a stack?
what is the point of having a seperate Stack class?
...isn't it literally just a less powerful ArrayList?

well, Java loooves classes



and Python favors *elegance*

5.1.1. Using Lists as Stacks

The list methods make it very easy to use a list as a stack, where the last element added is the first element retrieved ("last-in, first-out"). To add an item to the top of the stack, use `append()`. To retrieve an item from the top of the stack, use `pop()` without an explicit index. For example:

```
>>> stack = [3, 4, 5]
>>> stack.append(6)
>>> stack.append(7)
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack
[3, 4, 5, 6]
>>> stack.pop()
6
>>> stack.pop()
5
>>> stack
[3, 4]
```



